

Resumen del capítulo: Diseñar y desarrollar dashboards con Dash

Dashboards

Un **dashboard** (cuadro de mandos o tablero) es un informe interactivo que refleja un conjunto de métricas de negocio esenciales para administrar una empresa y se actualiza automáticamente.

- "Actualizado automáticamente": los datos utilizados para crear un dashboard se actualizan periódicamente (cada día/hora/minuto). Es por eso que la creación de dashboards está muy relacionada con la automatización.
- "Interactivo": los dashboards a menudo tienen controles para filtrar la información que se muestra. Generalmente, los datos se filtran por tiempo.
- "Conjunto de métricas de negocio": los dashboards muestran la información necesaria para resolver problemas comerciales..
- "Para la gestión de una empresa": los dashboards se utilizan para tomar decisiones empresariales.

Los paneles se pueden crear en varios sistemas como Tableau, QlikView, Microsoft Power BI y Amazon QuickSight. Estos productos son muy similares y no requieren ningún código adicional pero son bastante caros. Google Data Studio es una opción gratuita pero su funcionalidad es limitada. Para evitar pagar por sistemas comerciales sin limitar nuestras capacidades de creación de dashboards, utilizaremos la librería **Dash** en Python.

Dash es un conjunto de librerías de Python para crear dashboards y mostrarlos en navegadores web. Las librerías Dash y plotly se encargan de trazar los dashboards mientras que el microframework **Flask**, un conjunto de librerías básicas para crear aplicaciones web, se encarga de mostrarlos en los navegadores.

Este es un ejemplo de código para un dashboard simple creado en Dash:

```
#!/usr/bin/python

import dash
import dash_core_components as dcc
import dash_html_components as html

import plotly.graph_objs as go

import pandas as pd

# definir el diseño
```

```

external_stylesheets = ['https://codepen.io/chriddyp/pen/bWLwgP.css']
app = dash.Dash(__name__, external_stylesheets=external_stylesheets)
app.layout = html.Div(children=[

    # encabezado
    html.H1(children = 'Linear function'),

    # gráfico
    dcc.Graph(
        figure = {
            'data': [go.Scatter(x = pd.Series(range(-100, 100, 1)),
                                y = pd.Series(range(-100, 100, 1)),
                                mode = 'lines',
                                name = 'linear_func')],
            'layout': go.Layout(xaxis = {'title': 'x'},
                                yaxis = {'title': 'y'})
        },
        id = 'linear_func_id'
    ),

])

# lógica del dashboard
if __name__ == '__main__':
    app.run_server(debug=True)

```

El diseño de un dashboard es la parte gráfica que mostrará todos los gráficos y controles de dashboard en la página.

Lanzar un dashboard en tu máquina local

Cómo lanzar un dashboard en tu máquina local en una línea de comando Linux:

1. Abre la línea de comando Linux.
2. Crea un dashboard simple y guárdalo como `dash_test.py`.
3. Instala Dash. Para hacerlo, necesitarás desplegar la distribución **pip**. Ejecuta los siguientes comandos de uno en uno:

```

sudo apt update
sudo apt install python3-pip # responde sí a todas las preguntas

```

El primer comando actualiza la librería con apps disponibles. En Linux las apps se instalan en la librería `apt`. Por defecto, `pip` no es parte de la `apt` librería `apt` así que tiene que ser actualizada.

El segundo comando `python3-pip` instala `pip` para Python3.

El comando `sudo` significa que los comandos que siguen deben ser ejecutados como administrador. Para ejecutar el comando `sudo` tendrás que introducir tu contraseña de usuario. Como norma, es la misma contraseña que utilizas para entrar al sistema.

Ejecuta el comando para instalar dash:

```
pip3 install dash==1.4.1
```

Habiendo hecho esto, ve al fichero que almacena `dash_test.py` y ejecuta el comando:

```
python3 dash_test.py
```

El sistema mostrará el siguiente resultado:

```
Running on http://127.0.0.1:8050/
Debugger PIN: 769-266-004
* Serving Flask app "test_local" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: on
Running on http://127.0.0.1:8050/
Debugger PIN: 849-234-596
```

Copia la línea <http://127.0.0.1:8050/> y abre este enlace en tu navegador. Verás una versión local de tu primer dashboard.

Lanzar un dashboard en una máquina virtual es más difícil. Además, necesitarás el conocimiento sobre cómo instalar y configurar los servidores web. En tu futuro trabajo, para desplegar dashboards necesitarás dirigirte a los administradores del sistema y decirles el código secreto: "Necesito desplegar una app Flask en el servidor Apache".

Recopilación de requisitos al crear un dashboard

La creación de dashboards requiere que te conviertas en analista de negocios o escritor técnico por un corto periodo. Tendrás que comunicarte con los clientes del dashboard, recopilar requisitos funcionales y redactar **requisitos técnicos**.

Antes de construir un dashboard, aclara los siguientes detalles con el cliente:

- Qué problema comercial se supone que debe resolver el dashboard y quién será el usuario principal
- Con qué frecuencia se espera que se use el dashboard
- Estructura de datos del dashboard: qué métricas (KPI o indicadores clave de rendimiento) debe mostrar, qué parámetros deben agruparse, qué parámetros deben usarse para formar cohortes de usuarios
- El tipo de datos que se mostrarán en el dashboard (valores absolutos, relativos o ambos)
- Las fuentes de datos

- La base de datos que almacenará los datos agregados
- Con qué frecuencia se deben actualizar los datos
- Qué gráficos debe mostrar y en qué orden
- Qué controles debe tener el dashboard

Idealmente, los clientes te brindan requisitos técnicos explícitos pero, en la vida real, los analistas a menudo tienen que redactarlos por su cuenta. En esta etapa debes encontrar respuestas a dos preguntas:

1. ¿Construir un dashboard es una buena manera de resolver este problema?
2. ¿Cuántos dashboards deberías crear?

Después averigua qué datos necesitas, qué métricas necesitan los dashboards para ilustrar, los tipos de gráficos que debería haber y qué controles y filtros necesitará el cliente.

Esboza un **borrador de dashboard** con el cliente, es decir, una especie de mapa o modelo que muestre los tipos de gráficos y sus tamaños y posiciones relativos. Puedes crear una tabla simple para tu borrador.

Una vez que hayas preparado un borrador del dashboard y los requisitos técnicos, es hora de comenzar con la parte técnica del proyecto. En esta etapa deberías preguntar a los administradores de la base de datos qué fuentes puedes utilizar. Decide qué bases de datos almacenarán los datos agregados y en qué servidor se ejecutará el script de tu dashboard.

Estos son los elementos básicos de cualquier dashboard:

- **Encabezado:** debes decirle al usuario lo que ilustra el dashboard
- **Descripción del dashboard:** un breve texto que describe el problema que resuelve el dashboard y notas sobre cómo funciona (si hay algo fuera de lo común)
- **Gráficos y diagramas**
- **Controles**

Creación de gráficos básicos en Dash

Importa `dash_html_components` como `html` :

```
import dash_html_components as html
```

La siguiente construcción agrega la etiqueta HTML H1 (encabezado de nivel superior) que contiene el nombre del dashboard:

```
html.H1(children = 'El encabezado del mejor dashboard hecho por el/la mejor analista del mundo')
```

Si necesitas añadir una imagen marcada con la etiqueta al dashboard, así es cómo:

```
html Img(src='image_address')
```

Y si necesitas texto, por ejemplo, un comentario sobre cómo funciona el dashboard o un encabezado para un control (la etiqueta <label>), usa `html.Label()`:

```
html.Label('Tu texto aquí')
```

Estudia los componentes de la librería `dash_html_components` aquí: <https://dash.plot.ly/dash-html-components>.

Importa la librería `dash_core_components` como `dcc`. Esta librería tiene los componentes más interesantes; son responsables de mostrar los gráficos y los controles del dashboard.

```
import dash_core_components as dcc
```

El gráfico principal del dashboard va dentro de `dcc.Graph()`:

```
dcc.Graph(
    figure = {
        'data': [go.Bar(x = max_urbanization['Entity'],
                        y = max_urbanization['Urban'],
                        name = 'max_urbanization')],
        'layout': go.Layout(xaxis = {'title': 'Country'},
                           yaxis = {'title': 'Max % of urban population'})
    },
    id = 'urbanization_by_year'
),
```

`dcc.Graph` tiene dos parámetros:

- `figure` describe el gráfico a visualizar.
- `id` es el nombre de gráfico único, que tú asignas. Más adelante aprenderemos a usar este parámetro para hacer gráficos interactivos.

Esto es lo que sucede dentro de `figure`:

```
figure = {
    'data': [go.Bar(x = max_urbanization['Entity'],
                    y = max_urbanization['Urban'],
                    name = 'max_urbanization')],
    'layout': go.Layout(xaxis = {'title': 'Country'},
                       yaxis = {'title': 'Max % of urban population'})
},
```

`figure` tiene dos características:

- `'data'` define el conjunto de gráficos que mostrará `dcc.Graph`. En este ejemplo agregamos un gráfico de barras (`go.Bar`), que tiene países a lo largo del eje X y el porcentaje máximo de urbanización a lo largo del eje Y. `go` es un enlace a la librería `plotly.graph_objs` (importar con `import plotly.graph_objs como go`). Esto te permite agregar cualquier diagrama de la biblioteca plotly a tu dashboard.
- `'layout'` establece parámetros para mostrar gráficos. Aquí debes indicar el componente `go.Layout` para mostrar las etiquetas de los ejes. En la mayoría de los casos no necesitarás nada más.

El contenido de `'data'` se establece de forma dinámica; es decir, puedes crear un conjunto de gráficos en cualquier parte del programa y pasarlo al parámetro `'data'`.

Así se dibujan gráficos básicos con la librería `plotly.graph_objs`.

go.Scatter dibuja:

- gráficos de líneas (`lines`)
- gráficos de áreas apiladas (`lines` con el parámetro `stackgroup = 'one'`)
- diagramas de dispersión(`markers`)

Los parámetros básicos de `go.Scatter` son:

- `x`, `y`: objetos `pd.Series` (por ejemplo, columnas `DataFrame`), que contienen valores a lo largo de los ejes X e Y
- `mode`: el modo de dibujo, donde indicas `lines`, `lines` con el parámetro `stackgroup = 'one'` o `markers`

go.Bar dibuja un gráfico de barras. Los parámetros básicos de `go.Bar` son:

- `x`, `y`: objetos `pd.Series` (por ejemplo, columnas `DataFrame`) que contienen valores para los ejes X e Y
- `barmode` define la forma en que se mostrarán dos o más datasets en un gráfico:
 - `barmode = 'stack'` mostrará uno encima del otro (gráfico de barras apiladas)
 - `barmode = 'group'` mostrará los datasets uno al lado del otro

Observa que `barmode` se indica en la sección `layout`.

go.Pie traza un gráfico circular. Los parámetros básicos de `go.Pie` son:

- `labels`: un objeto `pd.Series` que contiene los nombres de las categorías

- `values`: un `pd.Series` que contiene los valores de las categorías

go.Box traza diagramas de caja. El parámetro básico de `go.Box` es:

- `y`: un objeto `pd.Series` que contiene los valores de la variable requerida

go.Table traza tablas. Los parámetros básicos de `go.Table` son:

- `header`: el diccionario (dict) de encabezados de columna. Los valores del encabezado se pasan como una matriz dentro del parámetro `values`. Por ejemplo:

```
header = {'values': ['<b>Column 1</b>', '<b>Column 2</b>']}
```

Aquí `<b>` es una etiqueta HTML que pone el texto en negrita.

- `cells`: el diccionario (dict) de las celdas de la tabla. Los valores se pasan como una matriz dentro del parámetro `values`. Al **Invertir** o **transponer**, el `DataFrame` facilita el paso de sus valores a las `cells`. Esto se puede hacer con el operador `T`:

```
cells = {'values': your_data_frame.T.values}
```

Veamos un ejemplo simple de transposición. Tenemos un `DataFrame` llamado `df`:

<u>Aa</u> Nombre	Manzanas	Bananas	Queso	Naranjas
<u>Anna</u>	1	5	8	3
<u>Helen</u>	10	19	22	21
<u>Jackson</u>	45	34	99	44

Su versión transpuesta (`df.T`) se verá así:

<u>Aa</u> Nombre	<u>Anna</u>	<u>Helen</u>	<u>Jackson</u>
<u>Apples</u>	1	10	45
<u>Bananas</u>	5	19	34
<u>Cheese</u>	8	22	99
<u>Oranges</u>	3	21	44

Veamos cómo trabajar con una tabla.

```
#!/usr/bin/python

import dash
import dash_core_components as dcc
import dash_html_components as html
from dash.dependencies import Input, Output

import plotly.graph_objs as go

from datetime import datetime

import pandas as pd

# recuperar y transformar los datos
urbanization = pd.read_csv('data/urbanization.csv')
urbanization['Year'] = pd.to_datetime(urbanization['Year'], format = '%Y')

urbanization_table = urbanization.copy()
urbanization_table['Year'] = urbanization_table['Year'].dt.date
urbanization_table['Urban'] = urbanization_table['Urban'].round(2)

# diseño
external_stylesheets = ['<https://codepen.io/chriddyp/pen/bWLwgP.css>']
app = dash.Dash(__name__, external_stylesheets=external_stylesheets)
app.layout = html.Div(children=[

    # tabla de urbanización
    dcc.Graph(
        figure = {
            'data': [go.Table(header = {'values': ['<b>País o continente</b>',
                                                    '<b>Año</b>',
                                                    '<b>Max % de población urbana</b>'],
                                'fill_color': 'lightgrey',
                                'align': 'center'
                                },
                            cells = {'values': urbanization_table.T.values})],
            'layout': go.Layout(xaxis = {'title': 'País o continente'},
                                yaxis = {'title': 'Max % de población urbana'})
        ],
        id = 'urbanization_table'
    ),

])

if __name__ == '__main__':
    app.run_server(debug=True)
```

Y como resultado obtenemos la siguiente tabla:

País	Año	% de población urbana
Afganistán	1950-01-01	6
África	1950-01-01	14.28
Albania	1950-01-01	20.53
Argelia	1950-01-01	22.21
Samoa Americana	1950-01-01	61.77
Andorra	1950-01-01	38.8
Angola	1950-01-01	7.58
Anguila	1950-01-01	100
Antigua y Barbuda	1950-01-01	30.06
Argentina	1950-01-01	65.34
Armenia	1950-01-01	40.34
Aruba	1950-01-01	50.93

Además de los parámetros básicos, los componentes tienen otras configuraciones (como color de línea, etc.). Observa algunos ejemplos de cómo funcionan: <https://plot.ly/python/>

Para más inspiración, echa un vistazo a esta galería de visualizaciones hechas con Dash: <https://dash-gallery.plotly.host/Portal/>

Los fundamentos de trabajar con controles

Todo dashboard debe ser **interactivo**. En otras palabras, debe tener controles que permitan filtrar los datos que se muestran.

Aquí está el código de un dashboard:

```
#!/usr/bin/python

import dash
import dash_core_components as dcc
import dash_html_components as html

import plotly.graph_objs as go

from datetime import datetime

import pandas as pd

# recuperar y transformar datos
urbanization = pd.read_csv('data/urbanization.csv')
urbanization['Year'] = pd.to_datetime(urbanization['Year'], format = '%Y')
max_urbanization = (urbanization.groupby('Entity')
                    .agg({'Urban': 'max'})
                    .reset_index()
                    .sort_values(by = 'Urban', ascending = False)
                    .head(25))

# layout
external_stylesheets = ['<https://codepen.io/chriddyp/pen/bWLwgP.css>']
app = dash.Dash(__name__, external_stylesheets=external_stylesheets)
app.layout = html.Div(children=[

    # crear un encabezado con una etiqueta HTML
    html.H1(children = 'Max % of urban population'),
```

```

# seleccionar rango de tiempo
html.Label('Time range:'),
dcc.DatePickerRange(
    start_date = urbanization['Year'].dt.date.min(),
    end_date = urbanization['Year'].dt.date.max(),
    display_format = 'YYYY',
    id = 'dt_selector',
),

# gráfico de urbanización
dcc.Graph(
    figure = {
        'data': [go.Bar(x = max_urbanization['Entity'],
                        y = max_urbanization['Urban'],
                        name = 'max_urbanization')],
        'layout': go.Layout(xaxis = {'title': 'Country'},
                             yaxis = {'title': 'Max % of urban population'})
    },
    id = 'urbanization_by_year'
),
])

# lógica del dashboard

if __name__ == '__main__':
    app.run_server(debug=True)

```

Aquí está el control en sí:

```

# selección de rango de tiempo
html.Label('Time range:'),
dcc.DatePickerRange(
    start_date = urbanization['Year'].dt.date.min(),
    end_date = urbanization['Year'].dt.date.max(),
    display_format = 'YYYY',
    id = 'dt_selector',
),

```

Aquí `html.Label` establece el nombre de este control. `dcc.DatePickerRange`, el elemento en sí, es un cuadro de diálogo para seleccionar las fechas de inicio y finalización. El control `dcc.DatePickerRange` tiene cuatro parámetros:

- `start_date`: la fecha de inicio. De forma predeterminada, se establece en el año mínimo (primero) de observaciones: `urbanization['Year'].dt.date.min()`
- `end_date`: la fecha de finalización. De forma predeterminada, se establece en el año máximo (último) de observaciones: `urbanization['Year'].dt.date.max()`
- `display_format`: el formato de visualización de fecha/hora
- `id`: el identificador único del control

Crear un control es fácil: solo tienes que agregar el objeto que necesitas de la librería Dash al diseño e indicar sus parámetros e ID. Considera que los parámetros del control se forman dinámicamente sobre la base de los datos de entrada: `start_date` y `end_date` contienen los valores de fecha mínimo y máximo del DataFrame `urbanization`.

Controles básicos en Dash

Estos son los controles más populares:

- **DatePickerRange**—un elemento que selecciona el intervalo de tiempo para el que se muestran los datos
- **Checklist**—un elemento que selecciona categorías de datos (por ejemplo, un conjunto de países, géneros de juegos o tipos de transacciones)
- **Dropdown**—otro elemento que selecciona categorías de datos
- **Radioltems**—un elemento que selecciona una (y solo una) opción de un conjunto (por ejemplo, seleccionar el modo de visualización de un diagrama)

Seleccionar un rango de tiempo:

```
dcc.DatePickerRange(start_date = '2016',
                    end_date = '2019',
                    display_format = 'YYYY',
                    id = 'dt_selector', )
```

`start_date` y `end_date` son strings en el formato indicado en el parámetro `display_format`.

Puedes leer la documentación de este control en <https://dash.plot.ly/dash-core-components/daterangepicker>.

Las **listas de verificación** se utilizan cuando necesitas que sea posible elegir una o varias categorías:

```
dcc.Checklist( options = [{'label': 'Africa', 'value': 'afr'},
                        {'label': 'Eurasia', 'value': 'eur'},
                        {'label': 'Australia', 'value': 'au'},
                        {'label': 'Americas', 'value': 'am'}],
              value = ['afr', 'eur', 'au', 'am'],
              id = 'continent_selector' )
```

La matriz que contiene las opciones se proporciona en el parámetro `options`. Para cada opción, `label` es el valor que ve el usuario y `value` es su valor técnico para el procesamiento interno. De forma predeterminada, el parámetro `value` contiene los valores `value`.

Documentación: <https://dash.plot.ly/dash-core-components/checklist>

También se utiliza un menú desplegable cuando se necesita que sea posible elegir una o varias categorías:

```

dcc.Dropdown( options = [{'label': 'Africa', 'value': 'afr'},
                        {'label': 'Eurasia', 'value': 'eur'},
                        {'label': 'Australia', 'value': 'au'},
                        {'label': 'Americas', 'value': 'am'}],
              value = ['afr', 'eur', 'au', 'am'],
              multi = True, id = 'continent_selector' )

```

Los parámetros son similares a los de las listas de verificación.

El valor del parámetro `multi` determina si es posible seleccionar varios elementos de la lista.

Documentación: <https://dash.plot.ly/dash-core-components/dropdown>

Se utiliza un botón de radio cuando se debe seleccionar una (y solo una) opción:

```

dcc.RadioItems(options = [{'label': 'Dog', 'value': 'dog'},
                        {'label': 'Cat', 'value': 'cat'},
                        {'label': 'Possum', 'value': 'possum'}],
              value = 'possum',
              id = 'pet_selector')

```

La matriz que contiene las opciones se proporciona en el parámetro `options`. Para cada opción, `label` es el valor que ve el usuario y `value` es su valor técnico para el procesamiento interno. De forma predeterminada, el parámetro `value` contiene los valores `value`.

Documentación: <https://dash.plot.ly/dash-core-components/radioitems>

Puedes leer sobre otros tipos de controles en <https://dash.plot.ly/dash-core-components>

Controles e interactividad

El dashboard cuyo código se da abajo solo tiene un control: un botón de radio. Permite al usuario seleccionar la función a mostrar (el modo de visualización). Echa un vistazo al código del dashboard:

```

#!/usr/bin/python

import dash
import dash_core_components as dcc
import dash_html_components as html
import plotly.graph_objs as go

import pandas as pd

from dash.dependencies import Input, Output
import math

# diseño
external_stylesheets = ['<https://codepen.io/chriddyp/pen/bWLwgP.css>']

```

```

app = dash.Dash(__name__, external_stylesheets=external_stylesheets)
app.layout = html.Div(children=[

    # hacer un encabezado con una etiqueta HTML
    html.H1(children = 'Trigonometric functions'),

    # seleccionar el modo de visualización
    html.Label('Display mode:'),
    dcc.RadioItems(
        options = [
            {'label': 'Display sin(x)', 'value': 'sin'},
            {'label': 'Display cos(x)', 'value': 'cos'},
        ],
        value = 'sin',
        id = 'mode_selector'
    ),

    # gráfico
    dcc.Graph(
        # el parámetro figure se define de forma dinámica
        id = 'trig_func'
    ),
])

# lógica del dashboard
@app.callback(
    [Output('trig_func', 'figure'),
    ],
    [Input('mode_selector', 'value'),
    ])
def update_figures(selected_mode):

    #crear gráficos de acuerdo con el modo seleccionado
    x = range(-100, 100, 1)
    x = [x / 10 for x in x]
    y = [math.sin(x) for x in x]
    if selected_mode == 'cos':
        y = [math.cos(x) for x in x]
    data = [
        go.Scatter(x = pd.Series(x), y = pd.Series(y), mode = 'lines', name = selected_mode + '(x)'),
    ]

    # definir el resultado a mostrar
    return (
        {
            'data': data,
            'layout': go.Layout(xaxis = {'title': 'x'},
                                yaxis = {'title': 'y'})
        },
    )

if __name__ == '__main__':
    app.run_server(debug=True)

```

Importamos los componentes de la librería Dash que son responsables de las señales de control del dashboard. Las señales de control se generan cada vez que el usuario cambia uno de los controles.

```

from dash.dependencies import Input, Output

```

Además del encabezado, el diseño contiene otros dos elementos. El primero es:

```
html.Label('Display mode:'),
dcc.RadioItems(
    options = [
        {'label': 'Display sin(x)', 'value': 'sin'},
        {'label': 'Display cos(x)', 'value': 'cos'},
    ],
    value = 'sin',
    id = 'mode_selector'
),
```

Este define un botón de radio para seleccionar el modo de visualización. Y el segundo es:

```
# gráfico
dcc.Graph(
    # el parámetro figure se define de forma dinámica
    id = 'trig_func'
),
```

Este especifica el gráfico que se mostrará en el dashboard. Este gráfico no muestra nada en este momento ya que no tenemos el parámetro `figure`. Esto se debe a que nuestro dashboard definirá `figure` de manera dinámica en función de los valores que el usuario seleccione en los controles.

El diseño es seguido por este pequeño bloque de código:

```
# lógica del dashboard
@app.callback(
    [Output('trig_func', 'figure'),
    ],
    [Input('mode_selector', 'value'),
    ])
def update_figures(selected_mode):
    # código de actualización del gráfico
```

Esta es una función para procesar controles. Antes de estudiarla en detalle, vamos a ver un poco más de teoría.

Hasta ahora has estado trabajando con códigos de Python que se ejecutan de forma secuencial. Esta es una ejecución de comando **sincrónica**. Pero Dash usa un enfoque **asincrónico**. Aquí una parte del programa genera señales mientras que las otras partes las recogen y realizan acciones en consecuencia. Por ejemplo, cuando el usuario presiona un botón de radio, genera una señal.

`update_figures` siempre está esperando la señal y responde a ella.

```
# lógica del dashboard
@app.callback(
    [Output('trig_func', 'figure'),
    ],
    [Input('mode_selector', 'value'),
    ])
```

```
def update_figures(selected_mode):

    # los gráficos a visualizar, teniendo en cuenta los filtros
    x = range(-100, 100, 1)
    x = [x / 10 for x in x]
    y = [math.sin(x) for x in x]
    if selected_mode == 'cos':
        y = [math.cos(x) for x in x]
    data = [
        go.Scatter(x = pd.Series(x), y = pd.Series(y), mode = 'lines', name = selected_mode + '(x)'),
    ]

    # formar el resultado a mostrar
    return (
        {
            'data': data,
            'layout': go.Layout(xaxis = {'title': 'x'},
                                yaxis = {'title': 'y'})
        },
    )
```

Después hay algo llamado **decorador**:

```
@app.callback(
    [Output('trig_func', 'figure'),
    ],
    [Input('mode_selector', 'value'),
    ])
```

Básicamente, este código...

```
@app.callback(
    [Output('trig_func', 'figure'),
    ],
    [Input('mode_selector', 'value'),
    ])
def update_figures(selected_mode):
    ...
```

... le dice a la función `update_figures` qué controles le enviarán señales (`Input`) y qué elementos del dashboard debe actualizar después de que se ejecute la función (`Output`). En nuestro código de muestra `update_figures`:

- Está esperando una señal del botón de radio con `id = 'mode_selector'`. Cuando `update_figures` recibe una señal, tomará el parámetro `'value'` del botón de radio.
- Después de ejecutar, pasará su resultado al parámetro `'figure'` del gráfico `'trig_func'`. Así es como los gráficos del dashboard se actualizan de forma dinámica.

Aquí, el parámetro de entrada `selected_mode` obtendrá su valor de `Input('mode_selector', 'value')`. Por lo tanto, `selected_mode` será igual a `value` del control con `id = 'mode_selector'`.

¿Qué hacemos si varios controles generan señales para varios gráficos? Considera este caso:

```
@app.callback(
    [Output('plot_1', 'figure'),
     Output('plot_2', 'figure')],
    [Input('control_1', 'value'),
     Input('control_2', 'value'),
     Input('control_3', 'value')]
)
```

El dashboard tiene tres elementos, con los identificadores `control_1`, `control_2` y `control_3`, y dos gráficos con los identificadores `plot_1` y `plot_2`. `update_figures` debe definirse de la siguiente manera:`

```
def update_figures(control_1_value, control_2_value, control_3_value):
    # formar gráficos dinámicos
    return (
        { # plot_1 graph
          'data': plot_1_data,
          'layout': go.Layout(...)
        },
        { # plot_2 graph
          'data': plot_2_data,
          'layout': go.Layout(...)
        },
    )
```

El orden de los parámetros de input (entrada) y output (salida) se define por el orden de los elementos de `Input` y `Output` dados dentro del decorador.

Para organizar los elementos del dashboard de acuerdo con un borrador, debes aprender a trabajar con HTML. Anteriormente leíste sobre esto, en la sección del curso sobre análisis sintáctico. El proceso de trabajar con HTML se llama diseño.

Elementos del dashboard

Para organizar los elementos del dashboard de acuerdo con un borrador, debes aprender a trabajar con HTML.

De todas las etiquetas HTML, la que más nos interesa es `<div>` ("división"). En su forma más simple, esta etiqueta divide lógicamente el contenido de una página web en bloques, estos son los "ladrillos" con los que están hechas las aplicaciones.

Cada dashboard en Dash también es una página web. Dash muestra los elementos del dashboard en el framework **Bootstrap**. Puedes encontrar más información en

<https://www.w3schools.com/bootstrap/default.asp>

A los elementos HTML se les asignan clases, es decir, nombres especiales. Puedes definir una regla CSS general para los elementos que se encuentran dispersos por una página pero comparten una clase, y estos elementos se mostrarán de la misma manera. Nuestra tarea es asignar clases que mostrarán elementos `div` como una tabla y organizarlos en columnas y filas.

Necesitaremos clases de `'row'` y `'N columns'`. Estas tienen una librería CSS especial en Dash. En `'N columns'` N se sustituye por un número que representa el número de columnas (por ejemplo, `'three columns'` [tres columnas]).

Veamos el siguiente ejemplo para tener una mejor idea de cómo funcionan las clases. Este código HTML...

```
<div>
  <div>Una</div>
  <div>Dos</div>
  <div>Tres</div>
</div>
```

... mostrará el siguiente resultado en el navegador:

```
Una
Dos
Tres
```

Si aplicamos las clases Dash:

```
<div class = 'row'>
  <div class = 'four columns' >Una</div>
  <div class = 'four columns' >Dos</div>
  <div class = 'four columns' >Tres</div>
</div>
```

Recibiremos:

```
Una Dos Tres
```

Tres elementos div con la clase `'four columns'` (esto especifica el ancho; ver a continuación) se organizan en una fila, ya que se colocaron dentro de un elemento div con la clase de `'row'`. Así es como los elementos div se organizan en filas y columnas.

En Dash, el número de columnas en una fila está limitado a 12. Considera esto al especificar clases para columnas.

Puedes leer más sobre cómo trabajar con las clases Bootstrap en https://www.w3schools.com/bootstrap/bootstrap_grid_basic.asp.

Averigüemos cómo crear etiquetas `div` con las clases que necesitamos en Dash. Esto se puede hacer dentro del elemento `html.Div`. Escribamos un código de muestra con dos columnas:

```
html.Div([
    html.Div([
        html.Label('I use 9 out of 12 available columns:'),
    ], className = 'nine columns'),
    html.Div([
        html.Label('I use 3 out of 12 available columns:'),
    ], className = 'three columns'),
], className = 'row'),
```

Como puedes ver, podemos crear etiquetas HTML `div` en elementos `html.Div` dentro del diseño del dashboard. Las clases y el ancho se indican en el parámetro `className`.

Ahora aprendamos a controlar la altura de los elementos. La altura de `div` depende de la altura de su contenido, por lo que deberás establecer la altura de los gráficos para los que `div` sirve como contenedor. La altura se ajusta en el parámetro `style`. Se puede configurar con varias unidades de medida pero la forma más conveniente es usar una relativa, como el % del ancho de la página. Esta unidad se llama **vw** ("ancho de ventana gráfica"). Por ejemplo, si necesitas configurar la altura del gráfico al 25% del ancho de la página, escribe el siguiente código Dash:

```
dcc.Graph(
    style = {'height': '25vw'},
    id = 'sales_by_platform'
)
```

Finalmente, veamos algunos otros elementos de Dash que ayudarán a que tus dashboards sean más informativos y legibles.

`html.H1` muestra en la pantalla el encabezado del dashboard. Así es como se puede usar:

```
html.H1(children = 'I am a header')
```

`html.Label` muestra el texto en la pantalla:

```
html.Label('Lorem ipsum dolor sit amet, consectetur adipiscing elit.')
```

`html.Br` te permite agregar una línea vacía entre dos elementos del dashboard si los elementos están demasiado cerca uno del otro:

```
html.Br()
```